



۸۱۰۱۰۱۴۹۲



تمرین ۳

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی

سوال ۱

لایه انتقال، ارتباط منطقی و مجازی را بین پروتکل‌های مبدا و مقصد برقرار می‌کند. درواقع پیامی که از لایه بالاتر، یعنی لایه کاربرد را از طریق سوکت‌های مختلف می‌گیرد و با تکه تکه کردن (و یا تبدیل) آن به سگمنت و درواقع اضافه کردن هدر سگمنت به *payload*، شماره پورت مبدا و مقصد را در هدر سگمنت قرار می‌دهد و چون هر پروتکل به یک پورت نگاشت شده است، گیرنده نیز با دریافت سگمنت از روی شماره پورت مقصد می‌فهمد که به کدام پورت و تبعا سوکت مربوطه و کاربرد مربوطه در لایه بالاتر بدهد. در واقع در مبدا لایه انتقال با گرفتن پیام‌های مختلف و تجمیع آنها در فرایندی به اسم *multiplex* و همراه با تبدیل آنها به سگمنت و اضافه کردن پورت مبدا و مقصد به آن، سگمنت‌ها را به لایه شبکه (لایه زیرین) پاس می‌دهد. لایه شبکه مبدا نیز وظیفی انتقال بسته از *host* به *host* مقصد را بر عهده دارد. بعد از گرفتن پیام‌ها در لایه شبکه مقصد، لایه انتقال مقصد، سگمنت‌ها را از لایه شبکه مقصد می‌گیرد و پیام‌ها را طبق پورت مقصد درج شده در هدر (همراه با اطلاعات مربوطه با پورت و آدرس مبدا و مقصد و طبق پروتکل مربوطه در لایه انتقال)، عمل *demultiplex* را انجام داده و پیام را به سوکت مربوطه به آن پروتکل لایه کاربرد انتقال می‌دهد.

سوال ۲

۱.

مکانیزم *flow control* برای کنترل ازدحام در بافر گیرنده و کنترل حجم فضای خالی بافر گیرنده و جلوگیری از پر شدن بافر گیرنده است (که در غیر اینصورت می‌تواند باعث *loss* شدن پکت‌ها شود)

۲.

مکانیزم *congestion control* برای کنترل ازدحام در شبکه میانی *core* شبکه و روترهاست. درواقع از پر شدن بافرهای روترهای میانی شبکه (که باعث *loss* و *drop* شدن پکت‌ها شود) و مخصوصا لینک‌های *bottleneck* جلوگیری می‌کند.

در هر دو روش، در صورت پر بودن بافر گیرنده در *flow control* و یا بافر روتر های میانی در *congestion control*، سیگنال هایی در قالب *flag* در هدر سگمنت به فرستنده منتقل می‌شود تا با کاهش ساینز پنجره، نرخ ارسال فرستنده محدود و کاهش یابد.

مثلا گیرنده میتواند با ست کردن بیت $ECE = 1$ در پیام هدر سگمنت *ACK* به فرستنده، آن را از *congestion* در شبکه مطلع سازد.

سوال ۳

- یکی از پروتکل های لایه کاربرد بر اساس پروتکل *UDP*، پروتکل *SNMP* است. این پروتکل که مربوط به مدیریت شبکه است، در شرایط حاد شبکه و زمانی که در شبکه ازدحام داریم، لازم است عملیانی باشد و با ازدحام، نرخ خود را کم نکند و در شرایط حاد، کار کند. برای همین بهترست کنترل ازدحام نداشته باشد و به همین دلیل به *UDP* نیاز دارد.
- کاربرد های *streaming* و مولتی مدیا که حساس به نرخ هستند و یک حداقل پهنای باند پیوسته نیاز دارند و تغییرات نرخ را تحمل نمیکنند اما شاید بتوانند کمی *loss* را تحمل کنند، از پروتکل *UDP* در لایه انتقال استفاده میکنند.
- برای کاربرد و پروتکل *DNS* چون به سرعت باید آدرس مربوطه را *retrieve* کنند، از *UDP* برای تسریع این کار کمک میگیرند (به *connection setup* و یک *RTT* اضافه نیاز ندارد)

سوال ۴

۱.

این عمل که در فرستنده و مبدا، *data chunk* های مختلف را از سوکت های مختلف (که میتواند برای پورت های و پروسس های کاربرد های مخلف لایه کاربرد باشند) جمع‌آوری کنیم و هدر مربوطه را به هر کدام از آنها اضافه کنیم و به سگمنت هایی تبدیلشان کنیم و این سگمنت ها به لایه شبکه مبدا پاس بدهیم، به این عمل، *multiplex* میگویند.

۲.

اینکه در گیرنده و مقصد، این سگمنت ها را از لایه شبکه زیرین دریافت کرده و با مطالعه هدر هر سگمنت و فیلد های مختلف سگمنت، سوکت مربوطه را شناسایی کرده و دیتا را به سوکت مربوطه میدهد. این عمل که هر کدام از این پیام ها را به سوکت مربوطه به خودشان بدهد، عمل *demultiplex* نام دارد.

۳.

در پروتکل *UDP* به کمک آدرس *IP* و شماره پورت مقصد، سگمنت را به سوکت مربوطه در مقصد تحویل میدهد. در واقع وقتی که بسته به کمک آدرس *IP* مقصد به گیرنده رسید، لایه انتقال تنها طبق فیلد شماره پورت مقصد در سگمنت، دیتا را به سوکت مربوطه میدهد (و درواقع از شماره پورت مبدا و یا آدرس *IP* مبدا در این عمل *demultiplex* در پروتکل *UDP* استفاده نمیکند)

سوال ۵

۱.

شماره *seqNum* به شماره اولین بایت داده ارسالی اشاره میکند. مثلاً اگر داده از شماره بایت ۶۰ و به طول ۲۰ بایت باشد، *seqNum* آن ۶۰ خواهد بود.

۲.

گیرنده در پاسخ به پیام ارسال شده از سمت فرستنده و با دریافت داده مربوطه، در *ACKNum* خود، شماره شروع بایتی که منتظر دریافت آن است را میفرستد. درواقع این شماره نشان دهنده آن است که تا قبل آن بایت را دریافت کرده (یعنی به نوعی یک *ACK* تجمعی تا آن بایت را فرستاده) و از آن بایت به بعد را از کاربر میخواهد. مثلاً در مثال گفته شده، *ACKNum* را مساوی ۸۰ قرار میدهد که یعنی از بایت ۸۰ به بعد را میخواهد که می‌شود شروع بسته بعدی (البته در بعضی از انواع پیاده سازی‌ها، استاد گفتند که ممکن است شماره بسته ای را که دریافت کرده در شماره تاییدیه بگذارد اما به طور معمول و آنچه در درس و اسلاید گفته شد، شماره شروع سگمنت بعدی را میگذارد)

۳.

برای شماره توالی سگمنت بعدی باید از بایت $1000 + 100 = 1100$ شروع کند و برای همین *seqNum* سگمنت بعدی، ۱۱۰۰ است.

سوال ۶

۱.

شماره *seqNum* در هر کدام از بسته‌ها (که شماره اولین بایت آن است) به این شرح است:

بسته اول: ۴۰۲

بسته دوم: $۷۴۵ = ۳۴۳ + ۴۰۲$

بسته سوم: $۱۰۸۸ = ۳۴۳ + ۷۴۵$

بسته چهارم: $۱۴۳۱ = ۳۴۳ + ۱۰۸۸$

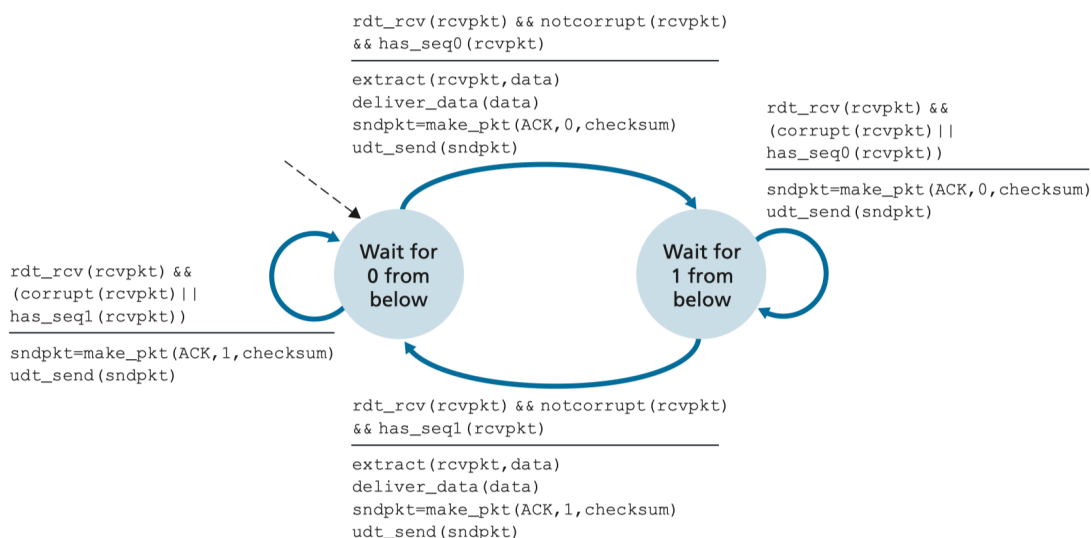
بسته پنجم: $۱۷۷۴ = ۳۴۳ + ۱۴۳۱$

۲.

از آنجایی که بسته اول *loss* میشود اما بسته دوم ارسال میشود اما گیرنده هنوز منتظر بسته اول بوده. پس شماره *ACK* آن، ۴۰۲ باقی خواهد ماند. بسته سوم و چهارم نیز *drop* می‌شود. بسته پنجم که ارسال شود، چون هنوز گیرنده *expectedSeqNum* همان ۴۰۲ است، در پاسخ *ACK* آن، دوباره شماره *ACK* را همان ۴۰۲ می‌گذارد یعنی هنوز منتظر دریافت ۴۰۲ است.

سوال ۷

همانطور که در شکل ۱ می‌بینید، *FSM* گیرنده در *rdt 3.0* تغییری نسبت به حالت قبلی *rdt 2.2* نکرده است. در *rdt 3.0* فقط تایمر اضافه شده که مربوطه به فرستنده است و نه گیرنده. عکس از کتاب مرجع است.



شکل ۱: *rdt 3.0* receiver FSM

سوال ۸

۱.

با توجه به راهنمایی اول گفته شده که میتوان لینک را تا اولین ACK دریافتی پر کرد، پس دیلی را همان RTT زمان رفت و برگشت میگیریم. پهنای باند هم در صورت سوال داده است. پس داریم:

$$BDP = R \times RTT = (10 \times 10^9) \times 0.6 = 6 \times 10^9 \text{ bits}$$

حال اگر این تعداد را به بسته های ۱۵۰۰ بیتی تقسیم کنیم، تعداد بسته های $in - flight$ را بدست خواهیم آورد و داریم:

$$Number \text{ of packets in flight} = \frac{6 \times 10^9}{1500 \times 8} = 5 \times 10^5 \text{ packets}$$

درواقع همانطور که گفته شد، مقدار BDP نشان دهنده حداکثر حجم داده‌ای است که می‌تواند در لوله ($Pipe$) ارتباطی جریان داشته باشد قبل از اینکه اولین تاییدیه (ACK) دریافت شود. در این سناریو، ۵۰۰,۰۰۰ بسته در هر لحظه می‌تواند در مسیر باشد. پس در نتیجه، برای اینکه از پهنای باند ۱۰ گیگابیت به طور کامل استفاده کنیم، همانطور که محاسبه کردیم، فرستنده باید پنجره‌ای ($WindowSize$) حداقل به اندازه ۵۰۰,۰۰۰ بسته داشته باشد. چون اگر پنجره کوچکتر باشد، لینک بیکار می‌ماند. [۱]

۲.

خب میدانیم که مشکل اصلی در GBN ، ارسال مجدد کل بسته های ACK نشده است، حتی اگر فقط یک بسته ACK نشده باشد، ممکن است لازم باشد کل بسته ها را دوباره از اول $retransmit$ کند. یعنی اگر یک $UNACKED$ داشته باشیم، حتی اگر بسته های بعدی هم فرستاده شده باشند، باز همه آن بسته ها از بسته ACK نشده دوباره باز ارسال می‌شوند.

حال اگر به احتمال یک ده هزارم، بسته ها $loss$ شوند،

از آنجایی که بیشتر اوقات باید این بسته های $retransmission$ را دوباره بفرستد، مقدار مفید $throughput$ بسیار کاهش میابد و چون مثلاً فرض کردیم که یک ده هزارم $loss$ داریم که حدوداً میشود ۵۰ بسته و پنجره ما هم در ابتدا ۵۰۰۰۰۰ است، پس احتمالاً بخاطر تایم اوت شدن، اندازه پنجره ریستارت شود و به ۱ برسد و حداکثر هم کمتر از ۱۰۰۰۰ خواهد بود چون دوباره با $loss$ و تایم اوت مواجه می‌شویم. پس همانطور که گفتیم بخاطر مصرف حجم زیادی از پهنای باند برای $retransmit$ کردن و همچنین تایم اوت شدن های پی در پی، $throughput$ موثر خیلی پایین تر از میزان ماکسیمم است. [۱]

۳.

چون در روش SR برعکس روش GBN ، بسته‌های خارج از ترتیب دور ریخته نمی‌شوند و تنها لازم است بسته‌های $loss$ شده که کلاً ۵۰ بسته در ۵۰۰۰۰۰ بسته بود، بازارسال شود (در مقابل GBN که کل بسته‌های بعد از بسته دراپ شده باید باز ارسال میشد) پس تعداد بسته‌های کمی باز ارسال می‌شود و بقیه بسته‌هایی که خارج از ترتیب به بافر مقصد رسیدند، نیاز به باز ارسال ندارند، و اینگونه از حداکثر میزان مفید پهنای باند توانسته است استفاده کند و باز ارسال‌های غیر ضرور نداریم، پس این روش برای لینک‌هایی مثل ماهواره با پهنای باند زیاد و تاخیر زیاد هستند، بسیار کارااست، و میتواند لوله را به طور بسیار موثری پر نگه دارد، [۱]

سوال ۹

اگر به طور ساده بخواهیم جواب دهیم، در صورتی که همین پروسس در $Host B$ به همان پروسس در $Host A$ پاسخ دهد و یا درخواستی داشته باشد، تنها کافی‌ست در هدر سگمنت، شماره پورت‌ها را جابجا کنیم، یعنی $source\ port\ number = y$ و $dest\ port\ number = x$ خواهد شد.

سوال ۱۰

برای این سوال، پیاده‌سازی مربوطه را به این شکل انجام می‌دهیم. شکل‌های ۲ و ۳ و ۴ کدهای مربوطه برای پروتکل $alternating - bit - protocol$ را نشان می‌دهند. در فایل `rdtsim.py` کد مربوطه به همراه توضیحات آن در فایل `explanation.txt` آمده است. [۲] حال توضیحات مربوط به منطق کد را اینجا می‌آوریم:

توضیحات منطق کد

- سمت A (فرستنده) همیشه فقط روی یک بیت seq کار می‌کند: یا ۰ یا ۱.
- وقتی داده‌ای با $seq = x$ می‌فرستد، منتظر $ACK(x)$ می‌ماند.
- اگر $ACK(x)$ سالم برسد $\rightarrow$ بسته تایید شده محسوب می‌شود و بیت را عوض می‌کند $(x \rightarrow 1 - x)$.
- اگر کنترل‌پکت خراب برسد یا $NACK(x)$ برسد یا تایم‌اوت شود $\rightarrow$ همان بسته $seq = x$ دوباره ارسال می‌شود.
- اگر $ACK/NACK$ با شماره‌ای غیر از x برسد $\rightarrow$ نادیده گرفته می‌شود.
- سمت B (گیرنده) یک متغیر دارد: `expected_seqnum` (در ابتدا ۰).
- اگر داده با $seq = expected_seqnum$ به‌طور سالم برسد:

به لایه ۵ تحویل داده می‌شود، $ACK(seq)$ همان شماره بسته ارسال می‌شود، مقدار $expected_seqnum$ تغییر می‌کند ($0 \leftrightarrow 1$).

• اگر بسته تکراری برسد (یعنی $seq \neq expected_seqnum$ ولی سالم باشد):

فقط $ACK(last_ok_seq)$ دوباره ارسال می‌شود، که $last_ok_seq = 1 - expected_seqnum$ است.

• اگر داده خراب برسد:

در نسخه فعلی کد، $NACK(expected_seqnum)$ ارسال می‌شود (شماره بسته‌ای که B انتظار دارد بعدی برسد).

خلاصه شماره‌گذاری:

ACK همیشه شماره آخرین بسته درست دریافت شده را می‌زند. در کد فعلی، $NACK$ شماره بسته مورد انتظار بعدی یعنی $expected_seqnum$ را می‌زند. A فقط $NACK$ هایی را جدی می‌گیرد که شماره آن‌ها $next_seqnum$ خودش باشد.

توضیح توابع $helper\ functions$ در خود کد ۲ آمده است. در ادامه قسمتی از کدهای $EntityA$ و $EntityB$ را توضیح می‌دهیم.

توضیح توابع $EntityB$ و $EntityA$

سمت A (فرستنده) ۲

• $send_new_packet(payload)$ این تابع یک بسته جدید را با شماره توالی فعلی $next_seqnum$ می‌سازد، چک‌سام آن را محاسبه می‌کند، آن را در $last_packet$ ذخیره می‌کند و با استفاده از to_layer3 به سمت B می‌فرستد. در پایان یک تایمر با مدت مشخص فعال می‌شود.

• $output(message)$ این تابع زمانی فراخوانی می‌شود که لایه ۵ داده جدیدی برای ارسال دارد. اگر A همچنان منتظر دریافت ACK باشد، پیام جدید نادیده گرفته می‌شود (رفتار پروتکل توقف انتظار). در غیر این صورت، پیام ورودی را به اندازه مناسب (۲۰ بایت) درآورده و از تابع $send_new_packet$ برای ارسال استفاده می‌کند.

• $input(packet)$ این تابع بسته کنترل ($ACK/NACK$) برگشتی از B را دریافت می‌کند. اگر هیچ بسته در حال انتظار وجود نداشته باشد، بسته کنترل نادیده گرفته می‌شود. اگر بسته خراب باشد، رفتار آن مانند $NACK$ خواهد بود: تایمر متوقف می‌شود و آخرین بسته ذخیره شده دوباره ارسال می‌گردد. اگر بسته سالم باشد:

• اگر ACK مربوط به شماره توالی فعلی باشد، تایمر متوقف شده، وضعیت انتظار پایان می‌یابد و شماره توالی از x به $1 - x$ تغییر می‌کند.

• اگر $NACK$ برای شماره فعلی باشد، بسته اخیر دوباره ارسال می‌شود.

• در غیر این صورت پیام کنترلی نادیده گرفته می‌شود.

• $timer_interrupt()$ در صورتی که تایمر A منقضی شود و همچنان منتظر ACK باشد، آخرین بسته

ارسال شده مجدداً فرستاده می‌شود و تایمر دوباره شروع می‌شود. این رفتار برای مقابله با ازدست‌رفتن بسته‌ها یا خرابی کنترل‌پکت‌هاست.

سمت B (گیرنده) ۴

- `__send_control(seqnum, is_ack)` این تابع یک بسته کنترل (*ACK* یا *NACK*) می‌سازد. در صورتی که `is_ack = true` باشد، *ACK* و در غیر این صورت *NACK* ارسال می‌شود. شماره کنترلی `acknum` روی `seqnum` تنظیم شده و بسته به لایه شبکه فرستاده می‌شود.
- `input(packet)` این تابع بسته داده‌ای را که از *A* به دست *B* رسیده مدیریت می‌کند:
- اگر بسته خراب باشد: *B* یک بسته `NACK(expected_seqnum)` ارسال می‌کند.
- اگر بسته دریافتی سالم بوده و شماره آن برابر `expected_seqnum` باشد: داده به لایه ۵ تحویل داده شده، سپس `ACK(seq)` فرستاده می‌شود و مقدار `expected_seqnum` تغییر می‌کند.
- اگر بسته سالم اما تکراری/غیرمنتظره باشد: بسته به لایه ۵ تحویل داده نمی‌شود، بلکه فقط `ACK(last_ok_seq)` تکراراً ارسال می‌شود.
- `timer_interrupt()` در پروتکل بیت‌متناوب، سمت *B* به تایمر نیازی ندارد؛ بنابراین این تابع فقط برای سازگاری با شبیه‌ساز وجود دارد و بدون عملکرد خاصی اجرا می‌شود.

در ادامه تصویری از خروجی کد `rdtsim.py` را مشاهده می‌کنید. ۵ توضیحات مربوط به خروجی، به دلیل سختی وارد کردن در لتک و حجم زیاد آن، در فایل `explanation.txt` قرار گرفته است.

بخش امتیازی، پروتکل GBN

کد مربوطه به این بخش در فایل `Bonus - gbn.py` قرار گرفته و خروجی نمونه آن و توضیحات خروجی آن در فایل `bonus - explanation.txt` قرار گرفته است. (به دلیل زیاد بودن توضیحات خروجی، از آوردن آن در این فایل لتک صرف نظر می‌کنم، لطفاً کد ها و توضیحات آن ها در فایل `txt` را بررسی فرمایید)

توضیح توابع EntityA و EntityB در پروتکل Go-Back-N

سمت A (فرستنده) ۶

سمت *A* در پروتکل Go-Back-N از یک `sliding window` استفاده می‌کند و می‌تواند تا اندازه ی پنجره چندین بسته را بدون دریافت *ACK* ارسال کند. *A* همچنین تمام بسته های ارسال شده را تا زمان دریافت یک *ACK* تجمعی

```
# ===== helper functions for checksum & control packets =====

def calc_checksum(seqnum, acknum, payload):
    """Compute checksum over header (seq/ack) and payload bytes."""
    s = seqnum + acknum
    for b in payload:
        s += b
    return s

def compute_pkt_checksum(pkt: Pkt):
    """Compute checksum corresponding to an existing Pkt object."""
    return calc_checksum(pkt.seqnum, pkt.acknum, pkt.payload)

def is_corrupted(pkt: Pkt) -> bool:
    """Return True if the packet checksum does not match."""
    return compute_pkt_checksum(pkt) != pkt.checksum

def make_control_payload(is_ack: bool) -> bytes:
    """
    Create a payload for control packets (ACK/NACK).
    First byte:
        'A' -> ACK
        'N' -> NACK
    Remaining bytes are zeros (to match Msg.MSG_SIZE = 20).
    """
    flag = ord('A') if is_ack else ord('N')
    return bytes([flag]) + bytes(Msg.MSG_SIZE - 1)

# ===== end helper functions =====
```

شکل ۲: helper functions

در بافر نگهداری می‌کند.

- `__send__packet(seq, payload)` این تابع بسته داده را با شماره توالی `seq` ساخته، چک‌سام آن را محاسبه کرده، در بافر فرستنده ذخیره می‌کند و سپس آن را با `to_layer3` به سمت `B` می‌فرستد. اگر این بسته همان بسته‌ای باشد که برابر با `base` است، تایمر آغاز می‌شود.
- `output(message)` این تابع زمانی صدا زده می‌شود که لایه ۵ یک پیام جدید برای ارسال ارائه می‌دهد. اگر پنجره پر نباشد، بسته جدید با شماره `nextseqnum` ساخته و ارسال می‌شود و سپس مقدار `nextseqnum` مدولار افزایش می‌یابد. اگر پنجره پر باشد، پیام داخل صف نگهداری قرار می‌گیرد (تا سقف مجاز).
- `input(packet)` این تابع بسته‌های ACK را از سمت `B` دریافت می‌کند. اگر بسته خراب باشد، نادیده گرفته

می‌شود (در GBN از $NACK$ استفاده نمی‌شود). اگر شماره ACK مربوط به یک بسته در پنجره فرستنده باشد، تمام بسته‌های تا آن شماره را به طور تجمعی تایید شده در نظر می‌گیرد و $base$ را جلو می‌برد. اگر پس از جلو بردن $base$ ، پنجره خالی شود، تایمر متوقف می‌شود؛ در غیر این صورت تایمر برای بسته جدید قدیمی‌تر ($new\ base$) دوباره آغاز می‌شود.

• **timer_interrupt()** اگر تایمر منقضی شود، A تمام بسته‌های موجود در محدوده $[base, nextseqnum)$ را (که بدون تایید مانده‌اند) دوباره ارسال می‌کند. سپس تایمر دوباره برای بسته جدید با شماره $base$ شروع می‌شود.

سمت B (گیرنده) ۷

سمت B فقط یک متغیر $expected_seqnum$ دارد و تنها بسته‌های دارای شماره توالی برابر با این مقدار را به عنوان «به موقع» و قابل تحویل به لایه ۵ قبول می‌کند.

• **send_ack(acknum)** این تابع یک ACK تجمعی با شماره $acknum$ می‌سازد و با استفاده از to_layer3 به فرستنده ارسال می‌کند. این شماره همیشه آخرین بسته درست و به ترتیب دریافت‌شده است.

• **input(packet)** رفتار گیرنده در GBN به صورت زیر است:

• اگر بسته خراب باشد: گیرنده هیچ داده‌ای را تحویل نمی‌دهد و فقط $ACK(last_ack)$ را دوباره ارسال می‌کند ($duplicate\ ACK$).

• اگر $packet.seqnum == expected_seqnum$ باشد: بسته «به ترتیب» است. داده به لایه ۵ تحویل داده می‌شود، $last_ack = expected_seqnum$ به‌روز می‌شود، $ACK(last_ack)$ ارسال می‌شود، و مقدار $expected_seqnum$ افزایش می‌یابد.

• اگر بسته سالم ولی خارج از ترتیب باشد: بسته تحویل نمی‌شود و تنها $ACK(last_ack)$ دوباره ارسال می‌شود.

• **timer_interrupt()** در پروتکل Go-Back-N سمت B هیچ تایمری ندارد و این تابع صرفاً برای سازگاری با شبیه‌ساز وجود دارد.

توضیحات مربوط به کد در قالب کامنت در خود کد آمده است. کلیات کد را می‌توانید در شکل ۶ و ۷ ببینید.

مراجع

[1] “bdr,” <https://gemini.google.com/share/29ba8cd0809f>, 2025.

[2] “alternating-bit-protocol,” <https://chatgpt.com/share/e/>, 2025.

```

class EntityA:
    def __init__(self, seqnum_limit):
        self.seqnum_limit = seqnum_limit
        self.next_seqnum = 0
        self.waiting_for_ack = False
        self.last_packet = None
        self.timeout_interval = 20.0

    def log(self, tag, msg=""):
        if TRACE > 0:
            print(f"[t={get_time(self):8.3f}] A | {tag:<16} {msg}")

    def _send_new_packet(self, payload_bytes: bytes):
        seq = self.next_seqnum
        checksum = calc_checksum(seq, 0, payload_bytes)
        pkt = Pkt(seq, 0, checksum, payload_bytes)

        self.last_packet = pkt
        self.waiting_for_ack = True

        to_layer3(self, pkt)
        start_timer(self, self.timeout_interval)

    def output(self, message: Msg):
        if self.waiting_for_ack:
            return

        payload = message.data
        if len(payload) != Msg.MSG_SIZE:
            payload = payload[:Msg.MSG_SIZE].ljust(Msg.MSG_SIZE, b'\0')

        self._send_new_packet(payload)

    def input(self, packet: Pkt):
        if not self.waiting_for_ack:
            return

        if is_corrupted(packet):
            stop_timer(self)
            to_layer3(self, self.last_packet)
            start_timer(self, self.timeout_interval)
            return

        flag = packet.payload[0]
        is_ack = (flag == ord('A'))
        is_nack = (flag == ord('N'))

        if is_ack and packet.acknum == self.next_seqnum:
            stop_timer(self)
            self.waiting_for_ack = False
            self.next_seqnum = 1 - self.next_seqnum
            return

        if is_nack and packet.acknum == self.next_seqnum:
            stop_timer(self)
            to_layer3(self, self.last_packet)
            start_timer(self, self.timeout_interval)
            return

    def timer_interrupt(self):
        if self.waiting_for_ack and self.last_packet:
            to_layer3(self, self.last_packet)
            start_timer(self, self.timeout_interval)

```

شکل ۳: Entity A code

```

class EntityB:
    def __init__(self, seqnum_limit):
        self.seqnum_limit = seqnum_limit
        self.expected_seqnum = 0

    # ----- Logging helper -----
    def log(self, tag, msg=""):
        if TRACE > 0:
            print(f"[t={get_time(self):8.3f}] B | {tag:<16} {msg}")

    # ----- Internal control sender -----
    def _send_control(self, seqnum: int, is_ack: bool):
        payload = make_control_payload(is_ack)
        checksum = calc_checksum(0, seqnum, payload)
        pkt = Pkt(0, seqnum, checksum, payload)

        kind = "ACK" if is_ack else "NACK"
        self.log(f"SEND_{kind}", f"seq={seqnum}")

        to_layer3(self, pkt)

    # ----- Receive data from A -----
    def input(self, packet: Pkt):
        self.log("RCV_DATA", f"seq={packet.seqnum}")

        if is_corrupted(packet):
            self.log("CORRUPTED", f"exp={self.expected_seqnum} → SEND_NACK")
            self._send_control(self.expected_seqnum, is_ack=False)
            return

        if packet.seqnum == self.expected_seqnum:
            self.log("IN_ORDER", f"seq={packet.seqnum} → deliver_to_L5")
            to_layer5(self, Msg(packet.payload))
            self._send_control(packet.seqnum, is_ack=True)
            self.expected_seqnum = 1 - self.expected_seqnum
            return

        # Duplicate / old seq
        last_ok = 1 - self.expected_seqnum
        self.log("DUPLICATE", f"seq={packet.seqnum} → resend_ACK({last_ok})")
        self._send_control(last_ok, is_ack=True)

    # ----- B does not use a timer -----
    def timer_interrupt(self):
        self.log("TIMER_UNUSED")

```

شکل ۴: Entity B code

```

===== SIMULATION BEGINS
[t=1787.107] A | SEND_DATA      seq=0 cks=1940
[t=1795.399] B | RECV_DATA      seq=0
[t=1795.399] B | IN_ORDER       seq=0 - deliver_to_L5
[t=1795.399] B | SEND_ACK       seq=0
TO_LAYER3: packet being corrupted
[t=1801.676] A | RECV_CTRL      acknum=0
[t=1801.676] A | CORRUPTED      action=retx_last
TO_LAYER3: packet being corrupted
[t=1809.168] B | RECV_DATA      seq=0
[t=1809.168] B | CORRUPTED      exp=1 - SEND_NACK
[t=1809.168] B | SEND_NACK      seq=1
[t=1810.215] A | RECV_CTRL      acknum=1
[t=1810.215] A | CTRL_IGNORED   acknum=1
[t=1821.676] A | TIMEOUT        pending=True
[t=1821.676] A | RETX_DATA      seq=0
[t=1828.692] B | RECV_DATA      seq=0
[t=1828.692] B | DUPLICATE      seq=0 - resend_ACK(0)
[t=1828.692] B | SEND_ACK       seq=0
TO_LAYER3: packet being corrupted
[t=1835.205] A | RECV_CTRL      acknum=0
[t=1835.205] A | CORRUPTED      action=retx_last
TO_LAYER3: packet being corrupted
[t=1838.963] B | RECV_DATA      seq=0
[t=1838.963] B | CORRUPTED      exp=1 - SEND_NACK
[t=1838.963] B | SEND_NACK      seq=1
[t=1843.162] A | RECV_CTRL      acknum=1
[t=1843.162] A | CTRL_IGNORED   acknum=1
[t=1855.205] A | TIMEOUT        pending=True
[t=1855.205] A | RETX_DATA      seq=0
[t=1857.790] B | RECV_DATA      seq=0
[t=1857.790] B | DUPLICATE      seq=0 - resend_ACK(0)
[t=1857.790] B | SEND_ACK       seq=0
[t=1861.576] A | RECV_CTRL      acknum=0
[t=1861.576] A | ACK_OK         seq=0
[t=3786.288] A | SEND_DATA      seq=1 cks=1961
[t=3790.541] B | RECV_DATA      seq=1
[t=3790.541] B | IN_ORDER       seq=1 - deliver_to_L5
[t=3790.541] B | SEND_ACK       seq=1
[t=3792.432] A | RECV_CTRL      acknum=1
[t=3792.432] A | ACK_OK         seq=1
[t=3933.956] A | SEND_DATA      seq=0 cks=1980
TO_LAYER3: packet being lost
[t=3953.956] A | TIMEOUT        pending=True
[t=3953.956] A | RETX_DATA      seq=0
[t=3961.871] B | RECV_DATA      seq=0
[t=3961.871] B | IN_ORDER       seq=0 - deliver_to_L5
[t=3961.871] B | SEND_ACK       seq=0
[t=3966.202] A | RECV_CTRL      acknum=0
[t=3966.202] A | ACK_OK         seq=0
[t=4773.282] A | SEND_DATA      seq=1 cks=2001
TO_LAYER3: packet being corrupted
[t=4777.090] B | RECV_DATA      seq=1
[t=4777.090] B | CORRUPTED      exp=1 - SEND_NACK
[t=4777.090] B | SEND_NACK      seq=1
TO_LAYER3: packet being corrupted
[t=4780.438] A | RECV_CTRL      acknum=1
[t=4780.438] A | CORRUPTED      action=retx_last
[t=4787.446] B | RECV_DATA      seq=1
[t=4787.446] B | IN_ORDER       seq=1 - deliver_to_L5
[t=4787.446] B | SEND_ACK       seq=1
[t=4794.762] A | RECV_CTRL      acknum=1
[t=4794.762] A | ACK_OK         seq=1
[t=5505.900] A | SEND_DATA      seq=0 cks=2020
[t=5509.479] B | RECV_DATA      seq=0
[t=5509.479] B | IN_ORDER       seq=0 - deliver_to_L5
[t=5509.479] B | SEND_ACK       seq=0
[t=5513.269] A | RECV_CTRL      acknum=0
[t=5513.269] A | ACK_OK         seq=0
[t=5683.692] A | SEND_DATA      seq=1 cks=2041
TO_LAYER3: packet being lost
===== SIMULATION ENDS

SIMULATION SUMMARY

```

شکل ۵: output of rdtSim.py

```

class EntityA:
    def __init__(self, seqnum_limit):
        self.seqnum_limit = seqnum_limit
        self.base = 0 # oldest unACKed seq
        self.nextseqnum = 0 # next seq to use for a new packet
        self.window_size = seqnum_limit // 2
        self.timeout_interval = 20.0
        self.buffer = {}
        self.app_layer_queue = []
    def log(self, tag, msg=""):
        if TRACE > 0:
            print("[t={get_time(self):8.3f}] A | {tag:<16} {msg}")
    def _window_is_full(self):
        dist = (self.nextseqnum - self.base + self.seqnum_limit) % self.seqnum_limit
        return dist >= self.window_size
    def _seq_in_window(self, seq):
        if self.base <= self.nextseqnum:
            return self.base <= seq < self.nextseqnum
        else:
            return seq >= self.base or seq < self.nextseqnum
    def _send_packet(self, seq, payload_bytes: bytes):
        checksum = calc_checksum(seq, 0, payload_bytes)
        pkt = Pkt(seq, 0, checksum, payload_bytes)
        self.buffer[seq] = pkt
        self.log("SEND_DATA", f"seq={seq} cks={checksum}")
        to_layer3(self, pkt)
        if self.base == self.nextseqnum:
            start_timer(self, self.timeout_interval)
    def _try_send_from_queue(self):
        while self.app_layer_queue and not self._window_is_full():
            msg = self.app_layer_queue.pop(0)
            payload = msg.data
            if len(payload) != Msg.MSG_SIZE:
                payload = payload[:Msg.MSG_SIZE].ljust(Msg.MSG_SIZE, b'\0')
            seq = self.nextseqnum
            self._send_packet(seq, payload)
            self.nextseqnum = (self.nextseqnum + 1) % self.seqnum_limit
    def output(self, message: Msg):
        if self._window_is_full():
            self.log("BUFFER_LS", "reason=window_full")
            self.app_layer_queue.append(message)
            return
        payload = message.data
        if len(payload) != Msg.MSG_SIZE:
            payload = payload[:Msg.MSG_SIZE].ljust(Msg.MSG_SIZE, b'\0')
        seq = self.nextseqnum
        self._send_packet(seq, payload)
        self.nextseqnum = (self.nextseqnum + 1) % self.seqnum_limit
    def input(self, packet: Pkt):
        self.log("RECV_CTRL", f"acknum={packet.acknum}")
        if is_corrupted(packet):
            self.log("CORRUPTED", "action=ignore")
            return
        flag = packet.payload[0]
        is_ack = (flag == ord('A'))
        if not is_ack:
            self.log("CTRL_IGNORED", "reason=non-ACK")
            return
        acknum = packet.acknum
        if not self._seq_in_window(acknum):
            self.log("ACK_IGNORED", f"acknum={acknum}")
            return
        old_base = self.base
        self.log("ACK_OK", f"acknum={acknum} base={self.base} next={self.nextseqnum}")
        seq = old_base
        while True:
            if seq in self.buffer:
                del self.buffer[seq]
                if seq == acknum:
                    break
            seq = (seq + 1) % self.seqnum_limit
        self.base = (acknum + 1) % self.seqnum_limit
        if self.base == self.nextseqnum:
            self.log("WINDOW_EMPTY", "")
            stop_timer(self)
        else:
            stop_timer(self)
            start_timer(self, self.timeout_interval)
        self._try_send_from_queue()
    def timer_interrupt(self):
        self.log("TIMEOUT", f"base={self.base} next={self.nextseqnum}")
        if self.base == self.nextseqnum:
            self.log("NO_OUTSTANDING", "")
            return
        seq = self.base
        while True:
            pkt = self.buffer.get(seq)
            if pkt is not None:
                self.log("RETX_DATA", f"seq={seq}")
                to_layer3(self, pkt)
                seq = (seq + 1) % self.seqnum_limit
            if seq == self.nextseqnum:
                break
        start_timer(self, self.timeout_interval)

```

شکل ۶: EntityA in GBN

```

class EntityB:
    def __init__(self, seqnum_limit):
        self.seqnum_limit = seqnum_limit
        self.expected_seqnum = 0

        # Last in-order seq successfully delivered (for cumulative ACK)
        # Initially: last_ack = expected_seqnum - 1 (mod seqnum_limit)
        self.last_ack = (self.expected_seqnum - 1 + self.seqnum_limit) % self.seqnum_limit

    # ----- Logging helper -----
    def log(self, tag, msg=""):
        if TRACE > 0:
            print(f"[t={get_time(self):8.3f}] B | {tag:<16} {msg}")

    def _send_ack(self, acknum: int):
        """
        Send cumulative ACK(acknum) to A.
        """
        payload = make_control_payload(is_ack=True) # 'A' flag
        checksum = calc_checksum(0, acknum, payload)
        pkt = Pkt(0, acknum, checksum, payload)

        self.log("SEND_ACK", f"acknum={acknum}")
        to_layer3(self, pkt)

    # ----- Receive data from A -----
    def input(self, packet: Pkt):
        """
        GBN receiver:
        - If correct and in-order: deliver to layer 5, ACK this seq, advance expected_seqnum.
        - If corrupted or out-of-order: resend ACK for last_ack (cumulative).
        """
        self.log("RCV_DATA", f"seq={packet.seqnum}")

        # 1) Corrupted packet: resend ACK for last_ack
        if is_corrupted(packet):
            self.log("CORRUPTED", f"exp={self.expected_seqnum} -> resend ACK({self.last_ack})")
            self._send_ack(self.last_ack)
            return

        seq = packet.seqnum

        # 2) In-order packet: seq == expected_seqnum
        if seq == self.expected_seqnum:
            self.log("IN_ORDER", f"seq={seq} -> deliver_to_L5")
            to_layer5(self, Msg(packet.payload))

            # Update last in-order seq and send ACK
            self.last_ack = seq
            self._send_ack(self.last_ack)

            # Expect next sequence number (modulo)
            self.expected_seqnum = (self.expected_seqnum + 1) % self.seqnum_limit

        # 3) Out-of-order or duplicate: do not deliver, just resend ACK(last_ack)
        else:
            self.log("OUT_OF_ORDER", f"seq={seq} exp={self.expected_seqnum} -> resend ACK({self.last_ack})")
            self._send_ack(self.last_ack)

    # ----- B does not use a timer in GBN -----
    def timer_interrupt(self):
        self.log("TIMER_UNUSED", "")

```

شکل ۷: EntityB in GBN